



OCP

OpenCognition Protocol

"Where Minds Meet Machines"

OCP Complete Git Repository

Here's the complete OCP repository, ready to extract into a Git repo. The file is organized with `=== FILE: <path> ===` delimiters — split on those and write each block to disk.

What's included (40+ files):

Root: README, LICENSE (MIT), CONTRIBUTING, CODE_OF_CONDUCT, SECURITY, GOVERNANCE

spec/ — Specification changelog (the full spec itself would be the artifact from the prior message)

schemas/ — 9 JSON Schemas (Draft 2020-12) covering OCPUMF, agent records, DID documents, bond records, embeddings, insights, model deltas, task requests, and task responses

sdk/python/ — Full reference SDK with `pyproject.toml` and 11 modules:

- `crypto.py` — Ed25519, X25519, AES-256-GCM, SHA-3, agent ID derivation
- `identity.py` — DID generation, DID Document building
- `messages.py` — OCPUMF builder with fluent API, signing, validation
- `knowledge.py` — All three knowledge types (embeddings, insights, model deltas)
- `trust.py` — Trust levels, vouching, bonds, trust score computation
- `pvl.py` — Privacy Validation Layer with PII scanning and all 5 rejection codes
- `transport.py` — WebSocket and HTTPS transports with auth handshake
- `registry.py` — Agent Registry client (register, discover, resolve)
- `consensus.py` — Full consensus round management with weighted voting
- `agent.py` — High-level Agent class tying everything together
- `exceptions.py` — Complete error hierarchy

node/ — Dockerfile, docker-compose.yml, and config.yml for running a local OCP node + registry

tests/ — 8 test files with ~50 test cases covering identity, messages, crypto, knowledge, trust, PVL, transport, and consensus

examples/ — 3 runnable examples (simple agent, knowledge sharing, multi-agent consensus)

Specification (§8.3) — A complete key recovery section covering the Shamir threshold scheme parameters, share generation, custodian model (bonded peers, org vaults, cold storage, recovery services), the full recovery procedure with sequence diagram, two new OCPUMF message types (`recovery_request`, `recovery_share_response`), secondary recovery keys, share expiry/rotation, and a security properties summary.

SDK (`ocp/recovery.py`) — Full implementation including $GF(2^8)$ arithmetic with precomputed log/exp tables, `split_secret()` and `reconstruct_secret()` using Lagrange interpolation, `RecoveryManager` class with share generation, encrypted share distribution (ECDH + AES-256-GCM per custodian), share decryption, and reconstruction with fingerprint verification.

Schema (`recovery-share.schema.json`) — JSON Schema for the recovery share record format.

Tests (`tests/test_recovery.py`) — 15 test cases covering correctness (3-of-5, 2-of-3, any-t-subset), edge cases (all zeros, all ones, single byte), parameter validation, the full `RecoveryManager` lifecycle, encrypted share transfer, wrong-key rejection, and fingerprint mismatch detection.

Example (`examples/key_recovery.py`) — Runnable end-to-end demo of the entire recovery lifecycle.

To get started:

bash

```
mkdir ocp-spec && cd ocp-spec
# Extract files from the artifact
git init
git add .
git commit -m "feat: OCP v1.0 — initial protocol release"
```

OCP Complete Python SDK

14 modules totaling roughly 3,000 lines of production-grade code. Here's what's in each module and what was added or expanded beyond the earlier repository files:

constants.py (new) — All protocol-level constants from the spec in one place: size limits, timeouts, rate limits, transport identifiers, HKDF info strings, default recovery parameters. Every magic number in the SDK now references this file.

exceptions.py — Expanded with `OCPConsensusError`, `OCPrecoveryError`, and proper `__repr__` methods across all exception classes.

crypto.py — Added `encrypt_payload()` and `decrypt_payload()` for full message-level encryption, `SigningKeyPair.from_private_bytes()` for key recovery, `generate_uuid_short()` for prefixed IDs, and AAD support on AES-GCM.

identity.py — Added `from_private_key()` class method for restoring identity after recovery, `DIDDocument.from_dict()` for deserialization, `rotate_signing_key()` and `rotate_encryption_key()` for key rotation, recovery key support in DID Documents, and a `validate_did()` helper.

messages.py — Added `MessageValidator` class with separate structural and cryptographic validation, `is_expired()` helper, `build_ack()` and `build_error()` convenience functions, and three new message types (`RECOVERY_REQUEST`, `RECOVERY_SHARE_RESPONSE`, `SHARE_REVOKE`).

knowledge.py — Added `EmbeddingVector.from_floats()` factory, reproducibility hashes on insight provenance, fuller validation on all three types, and expanded docstrings.

trust.py — Added `BondPermissions.from_dict()` and `intersect()` for permission negotiation, `Bond.revoke()`, `Bond.involves()`, `Bond.peer_of()`, `Vouch.revoke()`, `determine_trust_level()` function, and active/valid state properties throughout.

pv1.py — Added UK postcode and Japanese phone number patterns, depth-limited recursive string extraction, and more detailed rejection messages.

recovery.py — Added `build_recovery_request_payload()` for constructing recovery OCPUMF messages, share expiry tracking, and lazy $GF(2^8)$ table initialization.

transport.py — Added exponential backoff retry on HTTP, `on_message()` handler registration for WebSocket, `is_connected/session_id` properties, deadline-aware auth handshake with `asyncio.wait_for`, and structured logging.

registry.py — Added `deregister()`, `AgentRecord.to_dict()`, and HTTP/2 support.

consensus.py — Added `Vote.sign()`, deadline enforcement in `cast_vote()`, `result_payload()` for building broadcast messages, and `is_past_deadline` property.

agent.py — Added `trust_level` property, `broadcast()`, `vouch_for()`, `receive_vouch()`, `revoke_bond()`, `initiate_consensus()`, `recovery_manager` property, and structured logging throughout.

cli.py (new) — `ocp-keygen` command-line tool for generating identities.

compliance.py (new) — `ocp-compliance` self-test runner that validates identity, crypto, messages, PVL, recovery, and consensus in a single pass.

OCP Complete JSON Schemas

22 JSON Schema files covering every data structure in the OCP specification. Here's what's included and what was expanded beyond the earlier versions:

Core protocol: `ocpumf.schema.json` — Fully reworked with reusable `$defs` for `OCPAgentId`, `OCPAgentIdOrBroadcast`, and `Base64urlString` types, all 21 message types including the three recovery types, and ISO 639-1 language code validation.

Identity: `did-document.schema.json` — Added `VerificationMethod` with `purpose` field for recovery keys, `KeyRevocation` with `reason` enum (rotation/compromise/decommission), and `TrustAttestation` with domain and expiry support.

Trust: `bond-record.schema.json` — Added `revoked`, `revoked_by`, `revoked_at` fields, `allowed_task_types` on task delegation permissions, `max_epsilon` on model delta permissions, and `patternProperties` validation on signatures. `vouch-record.schema.json` (new) — Complete vouch record with conditional requirement for `revokes_vouch_id` when the vouch type is revocation.

Knowledge (3 schemas): `embedding.schema.json` — Added `model_version` to vector metadata. `insight.schema.json` — Added `severity` field, `importance` on features, `parent_insights` for provenance chains. `model-delta.schema.json` — Added `Compatibility` for base model version constraints, `clip_norm` and `accounting_method` to differential privacy, `training_rounds` and `data_distribution` to provenance.

Tasks (2 schemas): `task-request.schema.json` — Added `data_ref` for external input, `context`, `priority`, and `max_tokens` constraint. `task-response.schema.json` — Added `TaskProgress` with `percent/stage/ETA`, `retryable` on errors, `compute_units` on execution metadata, and conditional `allOf` rules requiring `result` when completed and `error` when failed/rejected.

Consensus (3 schemas, all new): `consensus-initiate.schema.json` — Full initiation payload with quorum config, eligibility criteria (trust level, domains, capabilities, invited agents), and context. `consensus-vote.schema.json` — Vote with rationale hash for auditability. `consensus-result.schema.json` — Result with vote summary.

Recovery (3 schemas): `recovery-share.schema.json` — Unchanged from earlier. `recovery-request.schema.json` (new) — Request payload with three proof types and conditional validation. `recovery-share-response.schema.json` (new) — Custodian response with encrypted share.

Transport (2 schemas, both new): `auth-handshake.schema.json` — WebSocket auth handshake frame. `auth-result.schema.json` — Server auth response with conditional `session_id/ttl` requirement on acceptance.

Registry (2 schemas, both new): `discover-request.schema.json` — Discovery request with sort, pagination, and extension filtering. `discover-response.schema.json` — Paginated response with `DiscoveryResult` items.

Errors (new): `error-response.schema.json` — Error payload with PVL code support, retry-after for rate limiting, and field-level error details.

Discovery (2 schemas, both new): `discovery-ping.schema.json` — Broadcast presence announcement. `capability-query.schema.json` and `capability-response.schema.json` — Query/response for capability introspection.

OCF Complete Reference Node – Docker Deployment

Complete OCF reference node — 10 source files plus Docker configuration, ready to build and run. Here's what each component does:

Infrastructure

`Dockerfile` — Multi-stage build that installs the Python SDK, node dependencies, and copies source/schemas/config. `docker-compose.yml` — Three services (transport node, registry, custodian) on an isolated bridge network with persistent volumes. `config.yml` — Every configurable parameter across all five protocol layers, rate limiting, cryptography, and the custodian.

Database (`database.py`)

A complete async SQLite persistence layer with 10 tables covering every OCF data structure: agents, bonds, vouches, tasks, consensus rounds, consensus votes, recovery shares, knowledge audit log, message deduplication, and trust interaction history. Each table has typed columns, indexes, and the database class exposes focused async methods for each domain (50+ methods total).

Message Router (`message_router.py`)

The core routing engine. Every inbound message passes through structural validation, deduplication (message_id tracking), TTL/expiry check, PVL enforcement for knowledge payloads (with audit logging), and dispatch to the appropriate handler. Returns structured OCF error responses on failure.

Handlers (`handlers.py`)

21 message handlers covering all OCPUMF message types: discovery ping, capability query, bond request/accept/revoke, knowledge share (with PVL pre-validated by the router), task request/response, consensus initiate/vote, recovery request, share revoke, ACK, broadcast, and error. The `register_all_handlers` function wires them all into the router.

Transport Server (`transport_server.py`)

Runs HTTP (Starlette/Uvicorn on port 8420) and WebSocket (websockets library on port 8421) concurrently. HTTP exposes the message endpoint plus REST APIs for querying bonds, tasks, and consensus rounds (including a resolve endpoint that tallies votes). WebSocket implements the full auth handshake with timeout, session tracking, and per-connection message routing.

Registry Server (`registry_server.py`)

Standalone API on port 8422 for agent registration, resolution, discovery (with domain/capability/trust filtering and pagination), vouch submission (with automatic Level 2 promotion when threshold is met), vouch queries, and network statistics.

Custodian Server (`custodian_server.py`)

Standalone API on port 8423 for encrypted Shamir share deposit, retrieval, revocation, and authenticated recovery. Runs a background cleanup loop that purges expired shares. Enforces capacity limits and proof type restrictions from config.

To run everything:

bash

`cd node`

`docker compose up -d`

Health checks

`curl http://localhost:8420/health` # Transport

`curl http://localhost:8422/health` # Registry

`curl http://localhost:8423/health` # Custodian

Register an agent

`curl -X POST http://localhost:8422/ocp/v1/registry/agents \`

`-H "Content-Type: application/json" \`

`-d '{"agent_id":"did:ocp:mainnet:agent-aabbccddeeff","display_name":"TestAI","capabilities":[{"id":"cap:nlp:classification","name":"Classification","version":"1.0"}],"domains":["research"],"endpoints":[{"transport":"ocp-http","url":"http://localhost:8420/ocp/v1/messages","priority":1}]}'`

Discover peers

`curl -X POST http://localhost:8422/ocp/v1/registry/discover \`

`-H "Content-Type: application/json" \`

`-d '{"filters":{"domains":["research"]}]}'`

Network stats

`curl http://localhost:8422/ocp/v1/registry/stats`